

API Security with Tyk: Policies in Tyk

Learn how Tyk manages API access, quotas, and rate limits through reusable, scalable templates.

Policies

- In Tyk, Security Policies act as templates for access control and rate limiting.
- Access Keys (tokens) are what clients use to authenticate and access APIs.
- By combining them, you can:
 - Standardize access control
 - Update access rights at scale
 - Implement tiered access levels (Basic → Premium → Enterprise)
 - Monitor and manage usage centrally

Security Policy Overview

- A Security Policy defines reusable rules for:
 - API Access lists
 - Rate limits
 - Quotas
 - Tags & metadata
- Acts as a **template** to apply consistent limits or permissions to many keys.
- Makes it easy to modify access for thousands of keys at once.

Why Use Policies?

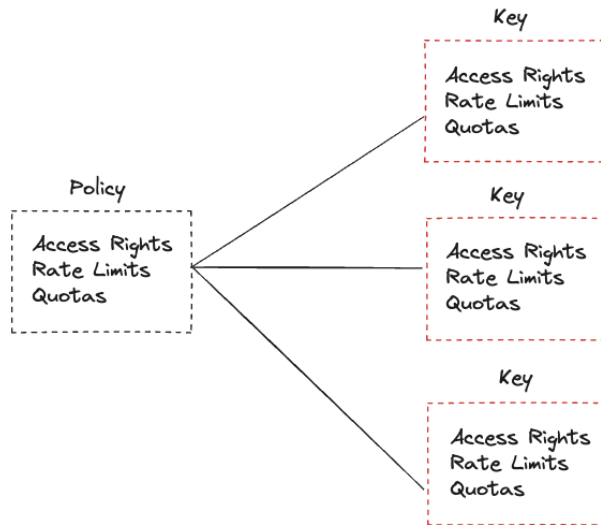
Without policies:

- Each API key must be updated manually.

With policies:

- Update once → all associated keys inherit changes.

Example: Upgrade 10,000 users' quotas by updating just one policy.



Policy Capabilities

A Tyk policy can define:

- APIs and versions accessible
- Allowed methods and paths
- Rate limits
- Quotas
- Tags and metadata

Policies can also be partitioned — e.g.

- Apply global ACLs for all users
- Apply per-user rate limits and quotas

Relationship: Policy vs Access Key

Concept	Description
Security Policy	A template that defines access control, rate limits, and quotas
Access Key	The actual token used by a client to access APIs

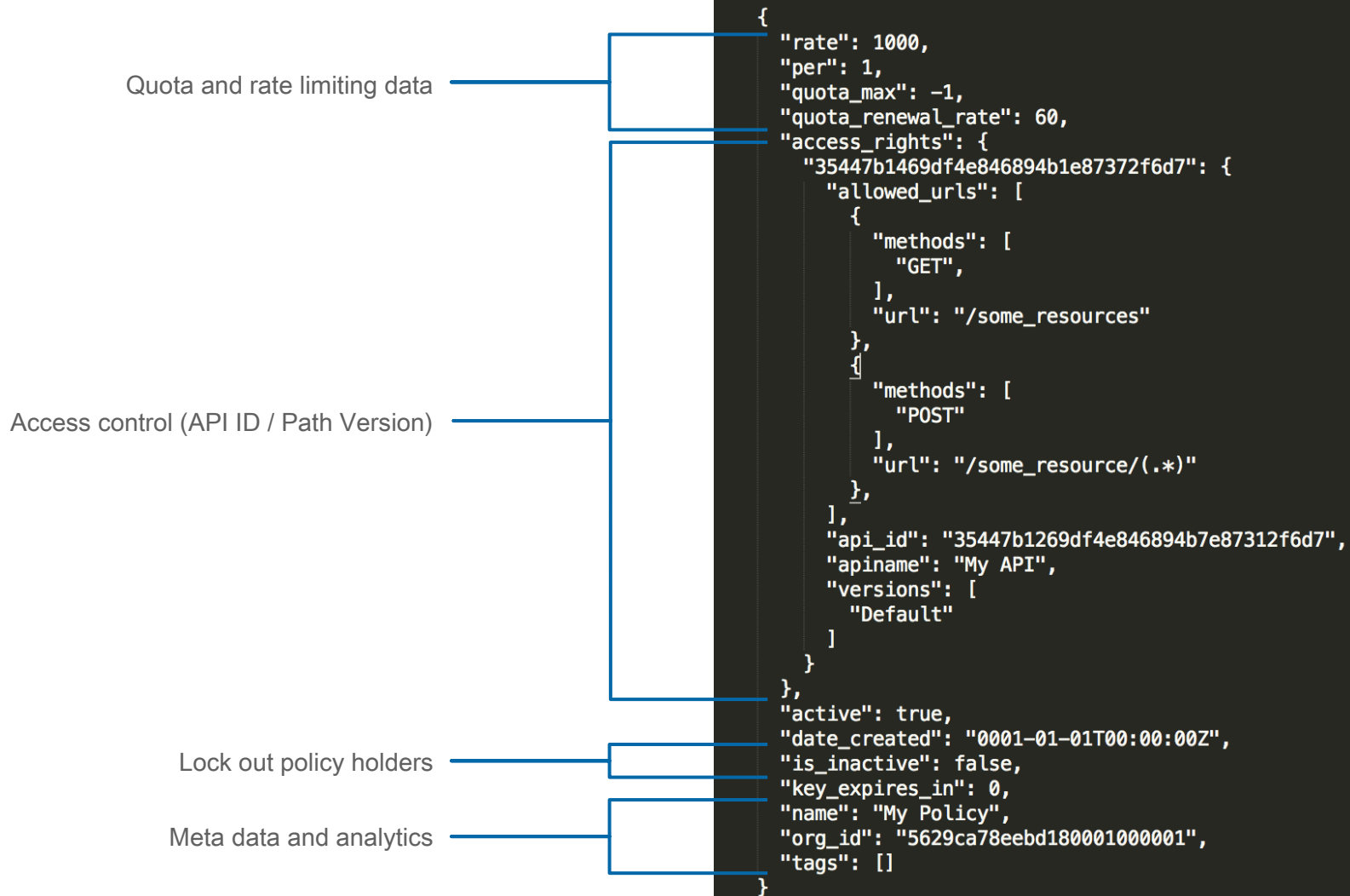
- Policies are applied to keys when keys are created.
- Policies override any individual settings on the key.
- Multiple policies can be combined for flexible control.

Policy Evaluation Flow

When a request comes in:

1. Tyk reads the Access Key
2. Checks which Policies are linked
3. Merges rules → applies rate limit, quota, ACL
4. Grants or denies API access accordingly

Note: This enables centralized management and consistent enforcement.



Key Policy Fields

Field	Description
<code>active</code>	Must be <code>true</code> for Tyk to load policy
<code>is_inactive</code>	Disables keys using this policy
<code>rate / per</code>	Rate limiting window
<code>quota_max / quota_renewal_rate</code>	Quota configuration
<code>access_rights</code>	Defines accessible APIs
<code>key_expires_in</code>	Optional trial key expiry time (in seconds)

Access Key Example

```
{  
  "org_id": "53ac07777cbb8c2d53000002",  
  "apply_policies": [  
    "59672779fa4387000129507d",  
    "53222349fa4387004324324e"  
  ],  
  "quota_max": 1000,  
  "rate": 3,  
  "per": 1  
}
```

The key inherits all limits, ACLs, and quotas from the applied policies.

Any changes to the policy → immediately reflected for this key.

Policies: General vs Per-API Rules

Rules defined in the policy can be set generally for all APIs, or specifically for individual APIs

Only some rules can be set per-API:

- Rate limiting
- Throttling
- Quotas
- Path-based permissions

Important: This cannot be used at the same time as Policy partitioning

Policies: Policy Partitioning

Policy partitioning allows policies to enforce only specific rules

Helps reduce the total number of Policies required and provide more flexibility

Policies must select at least one rule set to enforce:

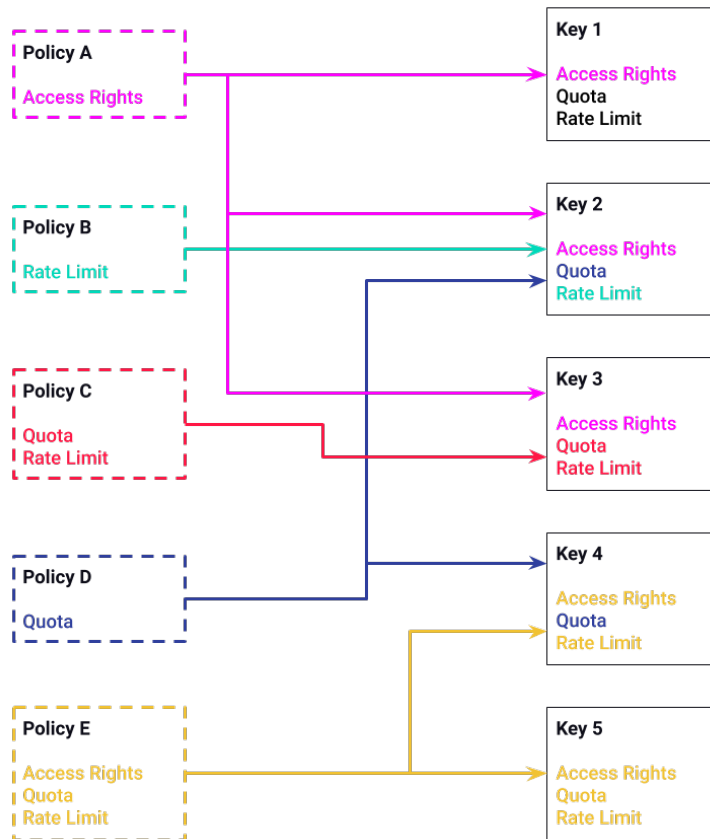
- Quotas
- Rate limiting
- Access control
- GraphQL query depth

Policies: Multi-Policy Keys

It is possible to assign multiple Policies to a Key:

- Policy rules will be merged to create a combined rule set
- It is possible to select Policies with the same API Access Rights, if their path-based permissions do not conflict
- When using partitioned Policies, their partitions must not overlap

Example



Example

